MREN 103 – Mechatronics and Robotics Design I
Mechatronics and Robotics Engineering Program
Queen's University, Kingston

Group #30
Stage #2 – Pickup and Drop-off Cycle

Student Names: Brady Cabral & Emily Vargas-Penaranda
Instructor Name: Prof. Surgenor
Date Report Submitted: April 5, 2026

Summary:
The objective of stage 2 was to combine information acquired from stage 1 and lab 6 to create a program and bucket system to transport parts doing two cycles. This is done by reusing the line following code and adding in control for the bucket mechanism to allow for retrieval and disposal of the parts. This allows for the Minibot to traverse between two points one with a pile of parts on one side and a bucket on the other side this is shown in **Fig. 1** and **Fig. 2** this is to be achieved twice consecutively then the MiniBot is to come to a stop

Fig 1. MiniBot retrieving parts from pile

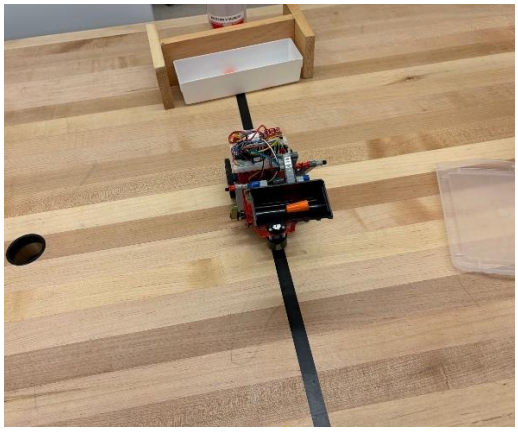

Fig 2. Minibot disposing parts into bucket

Hardware:
The MiniBot is controlled by the Arduino ItsyBitsy microcontroller, along with line-following sensors for navigation, a DC motor for the wheels, and a LEGO-based lifting arms and curling mechanism controlled by servomotors, as shown in **Fig. 3**. The track for stage 2 consists of a straight black tape, with wooden bins at each end (one for pickup and one for drop-off), spaced more than 75cm apart, shown in **Fig. 1** and **Fig. 2**.

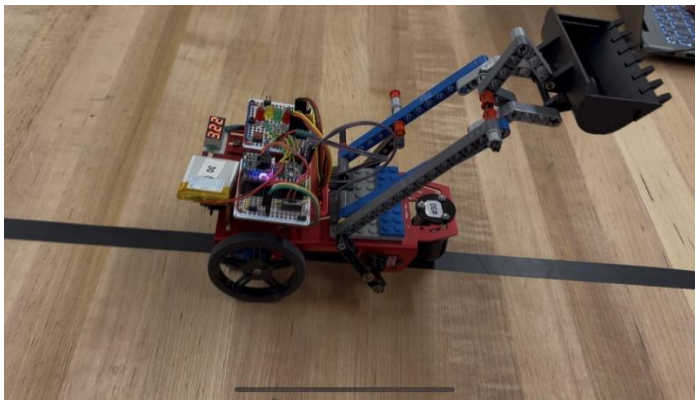

Fig 3. MiniBot with bucket mechanism

Software:
A listing of the program ***Group30ProjectLoaderStage2*** used in the lab is given in **Appendix A**.
The program is designed to get the Minibot to traverse from the pile of the parts and the bucket transporting at least 3 parts each time and to do this task twice in a row. This is done by breaking up the task into stages as shown in the program. There is the standard line following stage, which is from previous assignments including stage 1, this allows for the actual traversal of the MiniBot from the two points of interest. In the software the act of retrieving parts was broken up into multiple stages: TURN_AT_PICKUP, RECENTER_AFTER_PICKUP_TURN, ADJUST_AFTER_PICKUP_TURN, L OWER_BUCKET, BACK_UP_TO_COLLECT, RAISE_BUCKET. These stages allow for the robot to make a 180 degree turn when detecting a wall, moving forward to adjust, lowering the bucket by adjusting angles of servos, backing up the robot to collect the parts then raising the bucket back up then switching back to moving forward normally. The dispensing system of the MiniBot is controlled similar to the retrieval system as it broken into multiple smaller stages which are easier to debug: TURN_AT_DUMP, RECENTER_AFTER_DUMP_TURN, ADJUST_AFTER_DUMP_TURN, BACK_UP_TO_DUMP, DUMP_BUCKET, RESET_BUCKET. This breaks down the vague task of disposal into turning 180 degrees when a wall is detected and moving forward until detecting the line, then moving forward with the line following code for a short period of time to readjust angle then backing up to have the bucket above the disposal bin, next adjusting the curling servo to dump the contents of the bucket to then finally put the bucket back into its normal state to finally go back to traversing the line. The code also has a counter which increases after dumping the contents of the MiniBot, after this counter hits two the next time it comes within a certain distance of a wall it goes to a stop.

Results*:* The main difficulty experienced in task 2 was the scooping stage. Initially, the robot failed to scoop any parts during the pickup stage, as shown in **Fig. 4**. It went forward too much, even colliding with the wall. As a result, when curling the bucket in attempt to pick up the pieces, the bucket kept getting caught up with the wall, dropping all the pieces upward in a projectile fashion. To fix this, the pickup routine was slightly modified. With the bucket flat and in contact with the ground, the robot went forward, pushing pieces into the bucket. It then backed up sufficiently to have enough space between the bucket and the wall, to finally curl and lift the bucket, successfully scooping multiple pieces at a time. This allowed no interference between the bucket curl and the wall, preventing any unwanted loss of pieces.

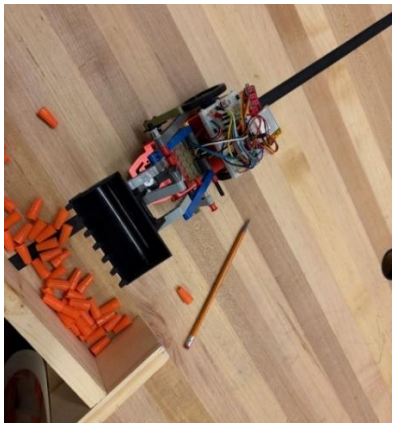

Fig 4. MiniBot failing to scoop parts

Conclusions:
In this stage, the line-following functionality developed in stage 1 was successfully combined with additional control logic for the pickup and drop-off mechanism to complete consecutive pickup and drop-off cycles, stopping autonomously after two cycles. This stage of the project highlighted the importance of code modularization, as condensing certain functionalities into helper functions allowed them to be easily reused throughout the robot's main routine (controlled by the switch statement covering each of its possible states). Additionally, this stage showed the need for iterative design, as multiple tests fine-tuning multiple variables were required to ensure a reliable performance.

Appendix A - Program Listing:

```
#include <Servo.h>

/*******************************************************
  Group30ProjectloaderStage2

  Original by B. Cabral & Emily Vargas-Penaranda
25/03/26

*******************************************************/

//pins
const int PIN_LED_RED    = 10;
const int PIN_LED_GREEN  = 9;
const int PIN_LED_YELLOW = 5;

const int PIN_BUTTON     = 7;
const int PIN_SHARP      = A3;
const int PIN_LINE_LEFT  = A2;
const int PIN_LINE_RIGHT = A1;

const int PIN_MOTOR_RIGHT = 3;
const int PIN_MOTOR_LEFT  = 4;

const int PIN_SERVO_LIFT = 11;
const int PIN_SERVO_CURL = 12;

//motion and sensor constants

const int STOP_PULSE_BASE = 149;    //neutral motor
pulse base (149 -> 1490 microseconds = stop)

const int LINE_THRESHOLD      = 2000; //sensor value
above this means the line is detected STNDRD 2300
const int LINE_THRESHOLD_R    = 1500; // sensor
value above this means the line is detected for right
sensor STNDRD 1500
const int FORWARD_SPEED       = 8;    //base forward
motor speed
const int FORWARD_OFFSET      = 0;    //small
correction offset to help the robot drive straighter
const int TURN_SPEED          = 10;   //motor speed
used when turning in place
const int RIGHT_TURN_OFFSET   = 5;    //extra
adjustment to make right turns more accurate

const int BACK_SPEED_RETRIEVE = -20;  //backward
speed used when collecting pellets
const int BACK_SPEED_DUMP     = -15;  //backward
speed used when moving into dump position
const int BACK_OFFSET         = 2;    //small correction
offset while moving backward

const int WALL_DETECT_NORMAL  = 1250; //distance
sensor threshold for detecting the wall during normal
cycles NRML : 1250
const int WALL_DETECT_FINAL   = 1250;  //distance
sensor threshold for detecting the final stopping wall

const unsigned long TURN_180_TIME_MS    = 1900;
//time needed to rotate about 180 degrees // HIGH :
1830 LOW : 2100 1950
const unsigned long POST_TURN_DELAY_MS  =
100;  //short pause after turning
const unsigned long PAUSE_TIME_MS       = 1000;
//general pause between major actions
const unsigned long SERVO_STEP_DELAY_MS =
20;   //delay between each small servo movement step
const unsigned long BACKUP_TIME_RETRIEVE_MS =
1300;  //time spent backing up to collect HIGH : 700
LOW : 1000
const unsigned long BACKUP_TIME_DUMP_MS =
1700; // time spent backing up to dump 1500

const unsigned long FORWARD_ADJUST_MS   =
800;  //time spent moving forward after recentering on
the line : 500

const int MAX_CYCLES = 2;

//servo angles

const int LIFT_ANGLE_DOWN = 100; // 135
const int LIFT_ANGLE_UP   = 38; // 60

const int CURL_ANGLE_OVERCURL = 65; // might
need to adjust
const int CURL_ANGLE_IN   = 60; // 58
const int CURL_ANGLE_OUT  = 52; // 90
const int CURL_ANGLE_DUMP = 38;

// dump value 38

//robot states
enum Stage {
  WAIT_START,
  MOVE_TO_PICKUP,
  TURN_AT_PICKUP,
  RECENTER_AFTER_PICKUP_TURN,
  ADJUST_AFTER_PICKUP_TURN,
  LOWER_BUCKET,
  BACK_UP_TO_COLLECT,
  RAISE_BUCKET,
  MOVE_TO_DUMP,
  TURN_AT_DUMP,
  RECENTER_AFTER_DUMP_TURN,
```

```
  ADJUST_AFTER_DUMP_TURN,
  BACK_UP_TO_DUMP,
  DUMP_BUCKET,
  RESET_BUCKET,
  FINISHED,
  TEST_MODE
};

//hardware objects
Servo leftWheel;
Servo rightWheel;
Servo liftServo;
Servo curlServo;

//global robot state
Stage stage = WAIT_START;

int cycleCount = 0;

int leftLineValue = 0;
int rightLineValue = 0;
int sharpRaw = 0;
int sharpMilliVolts = 0;

int currentLiftAngle = LIFT_ANGLE_UP;
int currentCurlAngle = CURL_ANGLE_IN;

unsigned long stageStartTime = 0;
bool started = false;


//helper functions

void detachBucketServos() {
  liftServo.detach();
  curlServo.detach();
}

void attachBucketServos() {
  liftServo.attach(PIN_SERVO_LIFT);
  curlServo.attach(PIN_SERVO_CURL);
}

//turns all LEDs ON or OFF at the same time
void setAllLEDs(bool value) {
  digitalWrite(PIN_LED_GREEN, value);
  digitalWrite(PIN_LED_YELLOW, value);
  digitalWrite(PIN_LED_RED, value);
}

//turns on only one LED (and turns the others off)
=>debugging purposes
void setSingleLED(int pin) {
  digitalWrite(PIN_LED_GREEN, LOW);
  digitalWrite(PIN_LED_YELLOW, LOW);
  digitalWrite(PIN_LED_RED, LOW);
  digitalWrite(pin, HIGH);
}

//stops both drive motors by sending the neutral pulse
void stopDrive() {
  int pulse = STOP_PULSE_BASE * 10; // convert base
to microseconds
  leftWheel.writeMicroseconds(pulse);
  rightWheel.writeMicroseconds(pulse);
}

//runs the motors with given speed adjustments (aka
deltas)
void runDrive(int leftDelta, int rightDelta) {
  int leftPulse = (STOP_PULSE_BASE + leftDelta) * 10;
  int rightPulse = (STOP_PULSE_BASE + rightDelta) *
10;

  leftWheel.writeMicroseconds(leftPulse);
  rightWheel.writeMicroseconds(rightPulse);
}

//reads all sensors and updates global variables
//line sensors are scaled for easier comparison
//distance sensor is converted to millivolts
void readSensors() {
  leftLineValue  =
map(analogRead(PIN_LINE_LEFT),  0, 1023, 0, 3000);
  rightLineValue =
map(analogRead(PIN_LINE_RIGHT), 0, 1023, 0,
3000);

  sharpRaw = analogRead(PIN_SHARP);
  sharpMilliVolts = map(sharpRaw, 0, 1023, 0, 3300);
}

//returns true if the left sensor detects the line
bool leftSeesLine() {
  return leftLineValue > LINE_THRESHOLD;
}

//returns true if the right sensor detects the line
bool rightSeesLine() {
  return rightLineValue > LINE_THRESHOLD_R;
}

//basic line-following logic:
//if right sensor sees line => turn right
//if left sensor sees line => turn left
//else => go straight
void followLineForward() {
```

```
  if (rightSeesLine()) {
    setSingleLED(PIN_LED_YELLOW);
    runDrive(FORWARD_SPEED, 0);
  }
  else if (leftSeesLine()) {
    setSingleLED(PIN_LED_RED);
    runDrive(0, FORWARD_SPEED);
  }
  else {
    setSingleLED(PIN_LED_GREEN);
    runDrive(FORWARD_SPEED, FORWARD_SPEED -
FORWARD_OFFSET);
  }
}

//moves servo smoothly from current angle to target
angle
void smoothMoveServo(Servo &servo, int
&currentAngle, int targetAngle) {
  if (currentAngle < targetAngle) {
    for (int angle = currentAngle; angle <= targetAngle;
angle++) {
      servo.write(angle);
      currentAngle = angle;
      delay(SERVO_STEP_DELAY_MS);
    }
  } else {
    for (int angle = currentAngle; angle >= targetAngle;
angle--) {
      servo.write(angle);
      currentAngle = angle;
      delay(SERVO_STEP_DELAY_MS);
    }
  }
}

//changes the robot's state and resets the timer for that
state
void beginStage(Stage nextStage) {
  stage = nextStage;
  stageStartTime = millis();
}

//returns how long the robot has been in the current
state
unsigned long timeInStage() {
  return millis() - stageStartTime;
}

//performs a 180-degree turn using timed motor control
//then transitions to the specified next stage
void performTurnAround(Stage nextStage) {
  stopDrive();
  delay(PAUSE_TIME_MS);
  setAllLEDs(HIGH);
  runDrive(TURN_SPEED, -TURN_SPEED -
RIGHT_TURN_OFFSET);
  delay(TURN_180_TIME_MS);

  stopDrive();
  setAllLEDs(LOW);
  delay(POST_TURN_DELAY_MS);

  //go back to forward motion after turning
  runDrive(FORWARD_SPEED, FORWARD_SPEED -
FORWARD_OFFSET);
  setSingleLED(PIN_LED_GREEN);

  beginStage(nextStage);
}

//lowers the lift and opens the bucket to prepare for
collecting pellets
void lowerBucketForCollection() {
  stopDrive();
  // attachBucketServos();
  setAllLEDs(LOW);
  delay(PAUSE_TIME_MS);

  smoothMoveServo(liftServo, currentLiftAngle,
LIFT_ANGLE_DOWN);
  smoothMoveServo(curlServo, currentCurlAngle,
CURL_ANGLE_OUT); // CURL_ANGLE_OUT

  beginStage(BACK_UP_TO_COLLECT);
}

//raises the lift and curls the bucket to secure collected
pellets
void raiseBucketAfterCollection() {
  delay(PAUSE_TIME_MS);
  setAllLEDs(HIGH);

  smoothMoveServo(curlServo, currentCurlAngle,
CURL_ANGLE_OVERCURL);
  delay(1000);
  runDrive(FORWARD_SPEED, FORWARD_SPEED -
FORWARD_OFFSET);
  delay(500);
  stopDrive();
  delay(50);
  smoothMoveServo(curlServo, currentCurlAngle,
CURL_ANGLE_IN);
  delay(1000);
  smoothMoveServo(liftServo, currentLiftAngle,
LIFT_ANGLE_UP);
```

```
  delay(1000);
  sharpMilliVolts = 0;
  setAllLEDs(LOW);
  beginStage(MOVE_TO_DUMP);
}

//opens the bucket to dump the pellets
void dumpBucket() {
  delay(PAUSE_TIME_MS);
  smoothMoveServo(curlServo, currentCurlAngle,
CURL_ANGLE_DUMP);

  beginStage(RESET_BUCKET);
}

//resets the bucket to its original position after dumping
//also increments cycle count and returns to pickup
phase
void resetBucketAfterDump() {
  delay(PAUSE_TIME_MS);
  smoothMoveServo(curlServo, currentCurlAngle,
CURL_ANGLE_IN);
  // detachBucketServos();
  cycleCount++;
  beginStage(MOVE_TO_PICKUP);
}

//handles backward motion for both collecting and
dumping
//moves backward for a fixed time, then transitions to
the next stage
void handleBackwardMotion(int speed, Stage
nextStage) {
  setSingleLED(PIN_LED_GREEN);
  runDrive(speed + BACK_OFFSET, speed);

  if ((stage == BACK_UP_TO_COLLECT &&
timeInStage() >= BACKUP_TIME_RETRIEVE_MS)
  || (stage == BACK_UP_TO_DUMP && timeInStage()
>= BACKUP_TIME_DUMP_MS)) {
    stopDrive();
    setAllLEDs(LOW);
    delay(SERVO_STEP_DELAY_MS);
    beginStage(nextStage);
  }
}

//setup
void setup() {
  pinMode(PIN_LED_GREEN, OUTPUT);
  pinMode(PIN_LED_YELLOW, OUTPUT);
  pinMode(PIN_LED_RED, OUTPUT);

  pinMode(PIN_BUTTON, INPUT);
  pinMode(PIN_SHARP, INPUT);
  pinMode(PIN_LINE_LEFT, INPUT);
  pinMode(PIN_LINE_RIGHT, INPUT);

  leftWheel.attach(PIN_MOTOR_LEFT);
  rightWheel.attach(PIN_MOTOR_RIGHT);

  liftServo.attach(PIN_SERVO_LIFT);
  curlServo.attach(PIN_SERVO_CURL);

  liftServo.write(currentLiftAngle);
  curlServo.write(currentCurlAngle);
  // 112 65
  // liftServo.write(LIFT_ANGLE_DOWN);
  // curlServo.write(53);
  // detachBucketServos();
  Serial.begin(9600);
  stopDrive();
}

//main loop
void loop() {
  readSensors();

//switch statement for each possible stage of the robot
//cleaner now with all the helper functions
  switch (stage) {

    case WAIT_START:
      if (!started) {
        while (digitalRead(PIN_BUTTON) == LOW) {
          digitalWrite(PIN_LED_GREEN, HIGH);
          delay(125);
          digitalWrite(PIN_LED_GREEN, LOW);
          delay(125);
        }
        started = true;
        beginStage(MOVE_TO_PICKUP);
      }
      break;

    case MOVE_TO_PICKUP:
    case MOVE_TO_DUMP:
      followLineForward();

      if (cycleCount == MAX_CYCLES && sharpMilliVolts
>= WALL_DETECT_FINAL) {
        stopDrive();
        beginStage(FINISHED);
      }
      else if (cycleCount < MAX_CYCLES &&
sharpMilliVolts >= WALL_DETECT_NORMAL) {
        if (stage == MOVE_TO_PICKUP) {
          beginStage(TURN_AT_PICKUP);
```

```
      } else {
        beginStage(TURN_AT_DUMP);
      }
    }
    break;

  case TURN_AT_PICKUP:
    performTurnAround(RECENTER_AFTER_PICKUP
_TURN);
    break;

  case TURN_AT_DUMP:
    performTurnAround(RECENTER_AFTER_DUMP_T
URN);
    break;

  case RECENTER_AFTER_PICKUP_TURN:
    setSingleLED(PIN_LED_GREEN);
    if (leftSeesLine() && !rightSeesLine()) {
      beginStage(ADJUST_AFTER_PICKUP_TURN);
    }
    break;

  case RECENTER_AFTER_DUMP_TURN:
    setSingleLED(PIN_LED_GREEN);
    if (leftSeesLine() && !rightSeesLine()) {
      beginStage(ADJUST_AFTER_DUMP_TURN);
    }
    break;

  case ADJUST_AFTER_PICKUP_TURN:
    followLineForward();
    if (timeInStage() >= FORWARD_ADJUST_MS) {
      beginStage(LOWER_BUCKET);
    }
    break;

  case ADJUST_AFTER_DUMP_TURN:
    followLineForward();
    if (timeInStage() >= FORWARD_ADJUST_MS) {
      beginStage(BACK_UP_TO_DUMP);
    }
    break;

  case LOWER_BUCKET:
    lowerBucketForCollection();
    break;

  case BACK_UP_TO_COLLECT:
    handleBackwardMotion(BACK_SPEED_RETRIEVE
, RAISE_BUCKET);
    break;

  case RAISE_BUCKET:
    raiseBucketAfterCollection();
    break;

  case BACK_UP_TO_DUMP:
    handleBackwardMotion(BACK_SPEED_DUMP,
DUMP_BUCKET);
    break;

  case DUMP_BUCKET:
    dumpBucket();
    break;

  case RESET_BUCKET:
    resetBucketAfterDump();
    break;

  case FINISHED:
    while (true) {
      setAllLEDs(HIGH);
      delay(125);
      setAllLEDs(LOW);
      delay(125);
    }
    break;

  case TEST_MODE:
    liftServo.write(LIFT_ANGLE_DOWN);
    curlServo.write(CURL_ANGLE_OUT);
    setSingleLED(PIN_LED_GREEN);
    runDrive(BACK_SPEED_RETRIEVE +
BACK_OFFSET, BACK_SPEED_RETRIEVE);
    break;
  }
}
```